

# ANÁLISIS DEL RETO

Sofía Ortiz Ossa, 2025020818, [s.ortizo2@uniandes.edu.co](mailto:s.ortizo2@uniandes.edu.co)

Valeria Ricaurte, 202520959, [v.ricaurte@uniandes.edu.co](mailto:v.ricaurte@uniandes.edu.co)

## Requerimiento 1

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrada</b>              | Catalog, brand                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Salidas</b>              | Número total de computadores con la marca requerida, promedio de precios, precio más alto, precio más bajo, promedio de ram, ram más alto, ram más bajo, promedio de vram, vram más alto, vram más bajo, promedio de núcleos de cpu, menor número de núcleos, mayor número de núcleos, año promedio de lanzamiento, menor año de lanzamiento, mayor año de lanzamiento, modelo de mayor precio con su precio respectivo, modelo de menor precio con su precio respectivo, tiempo de ejecución del programa. |
| <b>Implementado (Sí/No)</b> | Si . Sofía Ortiz                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

### Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos (todo se uso con funciones de array_list)                                                                    | Complejidad                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calcula el tamaño de lista_filtrada con los computadores de la marca dada por el usuario.                          | $O(n)$ recorre toda la lista para comparar que computadores cumplen con el filtro.                                                                                                                       |
| Hace una lista_filtrada de cada categoría únicamente con los computadores que tengan la marca dada por el usuario. | $O(n)$ Recorre el catalogo y compara cada marca con la marca dada por el usuario para varias categorías, sin embargo, $C * O(n) = O(n)$ .                                                                |
| En cada lista_filtrada, inserta los elementos según la categoría requerida.                                        | $O(1)$ ya que no es necesario agrandar el tamaño del array ya que esta predefinido (igual al número de computadores que cumplen con la marca requerida).                                                 |
| Calcular, promedio, mayor y menor de diferentes categorías.                                                        | $O(n)$ (en realidad $O(k)$ donde $k$ es el número de computadores filtrados) donde se recorre la nueva lista solo una vez para calcular tanto el promedio, el mayor y el menor de diferentes categorías. |

|                                                               |                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Acceder al modelo del computador con el mayor y menor precio. | $O(n)$ ) (en realidad $O(k)$ donde $k$ es el número de computadores filtrados) donde se recorre la nueva lista solo una vez para el mayor, y menor precio. Se usa <code>get_element</code> para acceder al modelo indicado, y para acceder al peso si es necesario ( $O(1)$ por lo tanto no aumenta la complejidad de este paso). |
| <b>TOTAL</b>                                                  | <b><math>O(N)</math></b>                                                                                                                                                                                                                                                                                                          |

## Análisis

En principio, se recorre la lista completa de computadores ( $O(n)$ ) para filtrar los computadores con la marca requerida. Sin embargo, después, para crear las listas\_filtradas de las categorías, se recorre solo  $O(k)$  donde  $k$  es el número de computadores con la marca requerida. Aunque se recorren varias veces la lista de marcas para hacer las listas de las categorías, el número de categorías \*  $O(k) = O(k)$ . Además, cuando se calcula el promedio, el máximo y el mínimo de una categoría, se recorre la lista una vez. Se aprovecha el `get_element` en el último paso para no aumentar la complejidad temporal de la última función.

| Año final | Tiempo /s |
|-----------|-----------|
| Samsung   | 0.015     |
| Apple     | 0.19      |
| Lenovo    | 0.126     |
| HP        | 0.12      |
| Acer      | 0.15      |
| Dell      | 0.19      |

## Requerimiento 2

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                             |                                                                                                                                                                                                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrada</b>              | Catalog, precio max, precio min                                                                                                                                                                             |
| <b>Salidas</b>              | Un diccionario con las llaves: "tiempo_ms", "cantidad", "promedio_ram", "promedio_vram", "promedio_precios", "mas_moderno", "menor_precio", "mayor_precio"<br>Si ninguno aplica para el rango, retorna None |
| <b>Implementado (Sí/No)</b> | Sí. Valeria Ricaurte                                                                                                                                                                                        |

## Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos                                                                                                                                           | Complejidad                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Recolecta en una SLL aquellos computadores que están entre el rango de precios. Cada computador es guardado en un nodo en forma de diccionario. | $O(n)$ ya que solo se hace un solo recorrido a la lista y se almacenan de una vez los computadores que cumplen con el rango                                                                                                                                                                                                                                           |
| Obtener los elementos de los computadores que sí aplican al rango de precios ( <code>sl.get_element()</code> )                                  | $O(1)$ como catalog comienza siendo un array list, el <code>get element</code> es de complejidad constante entonces no afecta el anterior $O(N)$ .                                                                                                                                                                                                                    |
| Agregar el nuevo diccionario a la SLL ( <code>sl.add_last()</code> )                                                                            | $O(1)$ ya que la estructura del SLL tiene el pointer al ultimo elemento de la lista.                                                                                                                                                                                                                                                                                  |
| Recorre la nueva SLL para encontrar los promedios, el más barato, el más caro, y el más moderno                                                 | $O(n)$ realmente es $O(k)$ donde $k$ es el numero de computadores que entran en el rango de precios. Es lineal ya que solo se recorre la SLL una sola vez, y a medida que avanza el recorrido se actualizan las sumas que se usaran en los promedios, y al mismo tiempo va actualizando el computador mas moderno/caro/barato. Se recorre en secuencia de nodo a nodo |
| <b>TOTAL</b>                                                                                                                                    | <b><math>O(N)</math></b>                                                                                                                                                                                                                                                                                                                                              |

## Análisis

La siguiente tabla muestra que la complejidad del Requerimiento 2 es efectivamente  $O(N)$ . Al analizarlo, esto ocurre ya que al llamar a los elementos del catalog, que es un `array_list`, la operación `get_element` es constante por cada elemento. De igual manera, se agregan los elementos que cumplen con el filtro a una nueva SLL; esto es eficiente en esta estructura de almacenamiento ya que agregar un elemento al final de la SLL tiene complejidad  $O(1)$ .

Pero, como realizamos este proceso de *evaluar+agregar* a toda la lista, se convierte en un proceso lineal.

De igual forma, para calcular las estadísticas de la nueva SLL, solo se recorre la lista una vez. En el peor de los casos, donde todos los computadores entran al rango de precios, el recorrido otra vez será  $O(N)$ . Lo cual resulta en un  $O(2N) \rightarrow O(N)$ .

| Precio Máx | Precio Min | Tiempo /ms |
|------------|------------|------------|
| 3000       | 2800       | 4418,20    |
| 3000       | 2600       | 5915,40    |
| 3000       | 2400       | 7499,30    |
| 3000       | 2200       | 9812,20    |

## Requerimiento 3

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                             |                                                       |
|-----------------------------|-------------------------------------------------------|
| <b>Entrada</b>              | Parámetros necesarios para resolver el requerimiento. |
| <b>Salidas</b>              | Respuesta esperada del algoritmo.                     |
| <b>Implementado (Sí/No)</b> | Si se implementó y quien lo hizo.                     |

### Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| <b>Pasos</b> | <b>Complejidad</b>         |
|--------------|----------------------------|
| Paso 1       | $O(...)$                   |
| Paso 2       | $O(...)$                   |
| Paso ....    | $O(...)$                   |
| <b>TOTAL</b> | <b><math>O(...)</math></b> |

### Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

## Requerimiento 4

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                             |                                                                                                                                                                                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrada</b>              | Parámetros necesarios para resolver el requerimiento.                                                                                                                                                                                                                                 |
| <b>Salidas</b>              | Número total de computadores con el cpu brand and gpu model requeridos, precio promedio, vram promedio, ram promedio, cpu_boost_ghz promedio de los computadores filtrados, marca, precio, año y cpu_model de los dos computdaores de mayor precio, tiempo de ejecución del programa. |
| <b>Implementado (Sí/No)</b> | Si. Sofía Ortiz.                                                                                                                                                                                                                                                                      |

### Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos                                                                                                      | Complejidad                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Se hacen las listas_filtradas de distintas categorías en formato SLL.                                      | $O(n^2)$ se recorre el catalogo primero para comparar si el elemento cumple con los requisitos de cpu_brand y gpu_model ( $O(2n) = O(n)$ ) requeridos por el usuario y luego se get_element de la categoría solicitada para insertarlo a la nueva lista (como se inserta a la nueva lista con add_last la complejidad temporal no aumenta más).                   |
| Se calcula el promedio de cada categoría basandose en su lista filtrada según las categorías..             | $O(n)$ (en realidad $O(k)$ donde k es el número de computadores que cumplen con los requisitos dados por el usuario). Se recorre toda la lista filtrada y se añaden los valores. Por último se accede al tamaño de la lista para dividir la suma total por la cantidad de valores (recorre la lista dos veces pero este paos no afecta porque $sl.size = O(1)$ ). |
| Se calcula el mayor precio usando la lista_filtrada de precios, dos veces, para el primer y segundo mayor. | $O(n)$ : Calcula el mayor precio de la lista recorriendo la lista entera. Luego vuelve a recorrer la lista para calcular el segundo mayor. ( $O(2n) = O(n)$ ).                                                                                                                                                                                                    |
| Eliminar elemento antes de calcular el segundo mayor precio.                                               | $O(n)$ elimina el primer mayor elemento de la lista de precios para calcular el sgundo mayor.                                                                                                                                                                                                                                                                     |
| Acceder a la información de los 2 computadores más costosos,                                               | $O(n)$ . Accede a la información de los computadores más costosos usando la función get_element.                                                                                                                                                                                                                                                                  |
| <b>TOTAL</b>                                                                                               | <b><math>O(n^2)</math></b>                                                                                                                                                                                                                                                                                                                                        |

## Análisis

El requisito tiene una complejidad de  $O(n^2)$ . Cuando se recorre el catalogo para filtrar los computadores con los requisitos del usuario, se usa la función get\_element para añadir los elementos de la categoría a la nueva single\_linked list. Esto hace que la complejidad temporal sea  $O(n) * O(n)$ . De resto, las otras funciones cuentan con una complejidad temporal de  $O(n)$ , donde simplemente se recorre la lista para acceder a la información necesaria.

| Cpu_brand | Gpu_model  | Tiempo /s |
|-----------|------------|-----------|
|           | Apple      |           |
| Apple     | Integrated | 0.099     |
| Intel     | RTX 40 70  | 0.013     |
| AMD       | Arc A580   | 0.011     |
|           | RX 5000 50 |           |
| AMD       |            | 0.17      |

## Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                             |                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrada</b>              | catalogo, filtro (BARATO o CARO), resolución, año mínimo de lanzamiento, año máximo de lanzamiento.                                                                                                                                                                                                                                                                       |
| <b>Salidas</b>              | Tiempo de la ejecución del requerimiento, Filtro de selección del costo, Número total de computadores que cumplieron el filtro (resolución y año), Información de precio, tamaño de pantalla, Gpu tier, tipo de display, año y peso del computador que cumpla con los requisitos, promedio de precio, tamaño y gpu_tier de los computadores que cumplan con el requisito. |
| <b>Implementado (Sí/No)</b> | Si. Sofia oRTIZ                                                                                                                                                                                                                                                                                                                                                           |

| Filtro | Resolución | Año_min | Año_max | Tiempo /s |
|--------|------------|---------|---------|-----------|
| Barato | 3840x2160  | 2000    | 3000    | 0.009     |
| Barato | 3840x2160  | 2010    | 3000    | 0.075     |
| Caro   | 3840x2160  | 2000    | 3000    | 0.011     |
| Caro   | 3840x2160  | 2010    | 3000    | 0.010     |

### Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos (funciones hechas con array_list)                     | Complejidad                                                                                                                                                               |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hallar el número de computadores que cumplen con el filtro. | $O(N)$ Recorre la lista del catalogo, revisa que computadores cumplen con el requisito.                                                                                   |
| Hace listas filtradas de las categorías requeridas.         | $O(n)$ (en verdad es $O(k)$ donde $k$ es el número de computadores que cumplen con el requisito), se recorren los computadores que cumplen con el filtro y se añaden a un |

|                                                                   |                                                                                                                                                                         |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                   | array (como el tamaño esta predeterminado, esto no le suma a la complejidad temporal).                                                                                  |
| Encontrar menor o mayor precio según el filtro ingresado.         | $O(n)$ , $O(k)$ donde $k$ es el número de computadores que cumplen con el requisito) donde recorre toda la lista de precios para encontrar al mayor ó al menor.         |
| Promedio de una categoría                                         | $O(n)$ , $O(k)$ donde $k$ es el número de computadores que cumplen con el requisito) se suman todos los valores de la lista y se dividen por el tamaño de la lista.     |
| Acceder a la información del computador con mayor ó menor precio. | $O(1)$ Ya con la posición del computador se acceden a las listas_filtradas de las categorías y se accede a la información de los computadores con menor o mayor precio. |
| <b>TOTAL</b>                                                      | <b><math>O(n)</math></b>                                                                                                                                                |

## Análisis

El requerimiento 5 comienza accediendo a los elementos usando `get_element` sobre el catálogo, lo cual es  $O(1)$  ya que es acceso directo. La función `lista_filtrada` recorre todos los elementos del catálogo con el fin de filtrar los que cumplen con la resolución proporcionada, al igual que los años. Como `get_element` y `insert_element` son  $O(1)$  y se recorre  $N$  veces, entonces en total será  $O(N)$ .

La función `lista_filtrada` se llama varias veces dentro de la función principal. Pero, como esto no ocurre dentro de un loop, mantiene su complejidad lineal.

Cuando las nuevas listas son creadas, tamaño  $M$ , se recorren estos nuevos elementos con el fin de conseguir las demás estadísticas requeridas por el problema. En el peor de los casos esto se hará  $N$  veces.

Algo que recalcar de esta función es que a pesar de ser lineal, tomará un poco mas de tiempo por el hecho de que se llama a la función `lista_filtrada` en repetidas ocasiones. Por ende la complejidad como tal sera  $O(tN + M)$ , donde  $t$  son las veces que se llama a la función auxiliar, y  $M$  son los elementos de la lista filtrada.

## Requerimiento 6

Plantilla para el documentar y analizar cada uno de los requerimientos.

### Descripción

Breve descripción de como abordaron la implementación del requerimiento

|                |                                                                                                                                                                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entrada</b> | catalog, año_inicial, año_final                                                                                                                                                                                                                          |
| <b>Salidas</b> | Un diccionario con las llaves: "tiempo_ms", "cantidad", "sist_op_mas_usado" (incluye el nombre, el total recuadado, y el total de registros), "sist_op_mas_recaudo" (incluye el nombre, el total recuadado, y el total de registros), "resumen_sist_op". |

|                             |                                               |
|-----------------------------|-----------------------------------------------|
|                             | Si ninguno aplica para el rango, retorna None |
| <b>Implementado (Sí/No)</b> | Si. Valeria                                   |

## Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos                                                                                                                                                                                                                                                                                                          | Complejidad                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paso 1: obtener tamaño de la lista y se crea una nueva lista                                                                                                                                                                                                                                                   | $O(1)$ , ya que es de acceso directo                                                                                                                                                                                                                                                                                                                                                                                                      |
| Paso 2: <code>for i in range (total)</code><br>filtrar los computadores dependiendo si están en el rango de años de lanzamiento y se agrupan de acuerdo con su sistema operativo.                                                                                                                              | $O(N)$ .<br>Las funciones por independiente ADENTRO DEL LOOP, como lo es <code>get_element</code> y <code>add_last</code> , son $O(1)$ . Pero, el loop como tal recorre toda la lista una sola vez para comprobar la condición así que será un $O(N)$                                                                                                                                                                                     |
| Paso 3: <code>for sist_op, lista in dicc_sist_op.items()</code><br>Calcula las estadísticas para cada uno de los sistemas operativos. Recorre cada computador por cada SO una sola vez. Como resultado, tenemos: total de equipos y recaudo; promedio de precio y de peso; el computador más costoso y barato. | $O(N) \sim O(\text{computadores que entran en el rango de años})$<br>El loop externo se encarga de recorrer los diferentes SO, acá se usa <code>size</code> que es $O(1)$ . Mientras que, el loop interno recorre cada computador de ese SO, acá hay un <code>get_element</code> que en el caso de array list es $O(1)$ . Entonces en total se evalúan $M$ computadores, donde $M$ son los computadores que resultaron del paso anterior. |
| <b>TOTAL</b>                                                                                                                                                                                                                                                                                                   | <b><math>O(N)</math></b>                                                                                                                                                                                                                                                                                                                                                                                                                  |

## Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Al comienzo de la función accede a los elementos uno a uno. Al ser array nos da complejidad de  $O(1)$  pues se utiliza la operación de `get_element`. En el primer loop, donde filtramos los elementos según el rango de años proporcionados, las funciones auxiliares que usamos como `get_element` y `add_last` son  $O(1)$ . Pero, al tener que evaluar cada elemento de la lista, este paso resulta siendo  $O(N)$ .

Ahora bien, Realizar las estadísticas de los computadores filtrados requiere un total de  $m$  pasos, donde  $m$  es el número de computadores que pasaron el filtro. Por lo tanto, en el peor de los casos donde todos los computadores pasan el filtro, la complejidad total de la función será de  $O(N+N) \rightarrow O(2N) \rightarrow O(N)$ .

| Año inicial | Año final | Tiempo /ms |
|-------------|-----------|------------|
| 2018        | 2019      | 6508,50    |
| 2018        | 2020      | 6789,20    |
| 2018        | 2021      | 9562,60    |



|      |      |          |
|------|------|----------|
| 2018 | 2022 | 9975,70  |
| 2018 | 2023 | 13698,00 |
| 2018 | 2024 | 17442,59 |

## Requerimiento Ejemplo

### Descripción

```
def get_data(data_structs, id):
    """
    Retorna un dato a partir de su ID
    """
    pos_data = lt.isPresent(data_structs["data"], id)
    if pos_data > 0:
        data = lt.getElement(data_structs["data"], pos_data)
        return data
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

|                             |                                                          |
|-----------------------------|----------------------------------------------------------|
| <b>Entrada</b>              | Estructuras de datos del modelo, ID.                     |
| <b>Salidas</b>              | El elemento con el ID dado, si no existe se retorna None |
| <b>Implementado (Sí/No)</b> | Si. Implementado por Juan Andrés Ariza                   |

### Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

| Pasos                                    | Complejidad              |
|------------------------------------------|--------------------------|
| Buscar si el elemento existe (isPresent) | $O(n)$                   |
| Obtener el elemento (getElement)         | $O(1)$                   |
| <b>TOTAL</b>                             | <b><math>O(n)</math></b> |

### Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal  $O(n)$ . Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.